

Contents

TripAdvisor Hotels — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	2
Your manager's request	3
Your team's job for the next 2 weeks (Module 3)	3
After Module 3	3
Why This Scenario is Special	3
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for EMMA)	4
Your plotting toolkit (you learned this in Module 2 Class 5)	4
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	5
Step 5 — Test it	6
Step 6 — Install a few NLP helpers	6
Colab tips	6
Class 1 — Data Cleaning	6
Your goal	6
Inputs	7
Outputs	7
Phase A — Explore the data first (15 minutes)	7
Phase B — Clean the reviews (45 minutes)	8
Phase C — Make ONE chart for Emma (15 minutes)	9
Common mistakes in Class 1	9
Self-check before Class 2	10
Class 2 — Encoding and Scaling	10
Your goal	10
Inputs	10
Outputs	10
Phase A — Explore the data first	10
Phase B — Encode and scale	11
Phase C — Make ONE chart for Emma	13
Common mistakes in Class 2	13
Self-check before Class 3	13
Class 3 — Feature Engineering	14
Your goal	14
Inputs	14
Outputs	14
Phase A — Explore the data first	14
Phase B — Engineer the features	15
Phase C — Make ONE chart for Emma	17
Common mistakes in Class 3	17

Self-check before Class 4	17
Class 4 — Feature Selection	18
Your goal	18
Inputs	18
Outputs	18
Phase A — Explore the data first	18
Phase B — Select features	18
Phase C — Make ONE chart for Emma	20
Common mistakes in Class 4	20
Self-check before Class 5	20
Class 5 — Pipelines	21
Your goal	21
Inputs	21
Outputs	21
Phase A — Explore the data first	21
Phase B — Build the pipeline	21
Phase C — Make ONE chart for Emma	23
Common mistakes in Class 5	23
Self-check before Class 6	23
Class 6 — End-to-End Lab (THE BIG ONE)	23
Your goal	23
Time	24
Inputs	24
Outputs	24
Phase A — Explore the data first (10 minutes)	24
Phase B — Build the final dataset (70 minutes)	24
Phase C — Findings report for Emma (10 minutes)	25
Grading rubric (100 points)	26
Submit	26
Tips for the whole module	26

TripAdvisor Hotels — Module 3 Lab Guide

Scenario: TripAdvisor Hotel Reviews. Predict the star rating from the text. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **TripAdvisor**. TripAdvisor is the biggest travel review website in the world. Tourists write reviews about hotels, restaurants, and tours.

Every month: - **Millions of travellers** read TripAdvisor before they book a hotel. - Hotels see their score and their text reviews. - Hotel managers must read the reviews and fix problems.

The problem

Hotels get **~100 reviews per month**. Some hotels in big cities get **1,000+ reviews per month**.

The hotel manager: - Has **zero time** to read all of them. - Misses common complaints (wifi slow, breakfast cold, room noisy). - Sees only the average score (4.2 out of 5) — and that does not tell her WHY guests are unhappy.

Result: small problems become big problems. Bad reviews continue. The hotel loses bookings.

Your manager’s request

Your manager, **Emma** (Head of Hotel Partnerships, EMEA), tells you:

“Our hotel managers get 100 reviews per month and zero time to read them.

Give me a tool. **Read each review. Auto-tag it with star rating + main complaint topic.** The manager opens the dashboard, sees: ‘this week, 12 reviews complained about wifi, 8 about breakfast.’

Then she fixes wifi on Monday morning. She fixes breakfast on Tuesday. Next month her score goes up.

This is the tool I want. Build it.”

Your team’s job for the next 2 weeks (Module 3)

Emma cannot do this alone. She has **one CSV file with 20,000 reviews**. The text is messy: HTML codes like & ; , smart quotes, duplicates, very short reviews, very long reviews.

Your job in Module 3: > **Turn one messy CSV file with raw English text into ONE clean file with 20+ engineered columns. The clean file will be used to train the model in Module 4.**

The clean file is called `tripadvisor_clean.parquet`. It must have **~21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the model. Predict the star rating (1 to 5) from the text.
Module 5	Find groups of reviews (very happy / very angry / mixed). For Emma’s dashboard.
Module 7	Real NLP: TF-IDF + sentiment + topic mining on the same Review column.

You use the **same TripAdvisor dataset** until the end of Module 7.

Why This Scenario is Special

Most data has many columns. This dataset has **only 2 columns**: Review (text) and Rating (1 to 5).

This is on purpose. It teaches you the most important lesson in modern AI:

When you only have text, you must MAKE the columns yourself.

This is called **text feature engineering**. You will count words, count exclamation marks, count capital letters, check if the word “wifi” appears, check if the word “not” appears, and so on.

Each count becomes one column. By the end of Module 3 you will have **20+ columns from text alone**. Then in Module 4, the model can learn from them.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for EMMA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Emma, look at this. This is the problem.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>

Plot	When to use it	Function name
Line plot	See change over time	<code>plt.plot()</code>
Word cloud	See most common words	<code>WordCloud().generate(text)</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything heavy. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `tripadvisor_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/tripadvisor_lab', exist_ok=True)
%cd /content/drive/MyDrive/tripadvisor_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

Option A — Direct from Kaggle (recommended):

1. Make a free Kaggle account.
2. Go to kaggle.com/settings/account, scroll to “API”, click “Create New Token”. Save the `kaggle.json` file.
3. In Colab:

```
from google.colab import files
files.upload() # pick kaggle.json
```

4. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d andrewmvd/trip-advisor-hotel-reviews
```

```
!unzip -q trip-advisor-hotel-reviews.zip -d data
!ls data/
```

You should see **1 CSV file** inside data/: tripadvisor_hotel_reviews.csv.

Option B — Upload by hand:

1. Download the zip from [kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews](https://www.kaggle.com/datasets/andrewmvd/trip-advisor-hotel-reviews) on your laptop.
2. Unzip it.
3. In Colab's file panel (left sidebar), upload the CSV file.

Step 5 — Test it

```
import pandas as pd
df = pd.read_csv('data/tripadvisor_hotel_reviews.csv')
print(df.shape)
print(df.columns.tolist())
print(df.head(2))
```

Should print (20491, 2) and the columns ['Review', 'Rating']. You are ready.

Step 6 — Install a few NLP helpers

You will use small libraries to count sentences and score sentiment. Install them once:

```
!pip install nltk vaderSentiment wordcloud -q
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

This takes about 30 seconds.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate .parquet files to Drive	Files in /content/ disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

Scenario reminder: Emma drops one messy CSV on your desk. The text has HTML codes like &;, smart quotes, duplicate reviews, and a few very short reviews ("Great!"). Your job today: clean the text and decide which rows to keep.

Your goal

Make the file USABLE. Fix the HTML. Remove duplicates. Drop empty / tiny reviews. Look at the rating distribution.

Inputs

- data/tripadvisor_hotel_reviews.csv

Outputs

- reviews_step1.parquet saved in your Drive folder
 - 3+ exploratory charts in your notebook
 - 1 explanatory chart for Emma
 - Notes in markdown about every decision you made
-

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many reviews per star rating?

- **Question:** “Of 20,491 reviews, how many are 5-star, how many 4-star, etc.?”
- **HINTS:**
 - Use `df['Rating'].value_counts().sort_index()`.
 - Then `.plot.bar()` on the result.
- **What you learn:** The data is **very unbalanced**. About 50% are 5-star and only ~5% are 1-star. Remember this — it will be a problem in Module 4.

Exploratory chart 2 — Length of the reviews

- **Question:** “Are reviews short (‘Great!’) or long (a full essay)?”
- **HINTS:**
 - First make a new column: `df['review_length'] = df['Review'].str.len()`.
 - Histogram with `bins=50`.
 - Try `plt.xlim(0, 4000)` to ignore very long outliers.
- **What you learn:** Most reviews are 500-2000 characters. A few are huge (10,000+).

Exploratory chart 3 — How many duplicate reviews?

- **Question:** “Did some users copy the same text twice?”
- **HINTS:**
 - Use `df['Review'].duplicated().sum()`.
 - Print the number.
 - Print a few of the duplicate texts: `df[df['Review'].duplicated(keep=False)].head(10)`.
- **What you learn:** Some reviews are template text (“Nice hotel, good staff.”). You will drop them.

Exploratory chart 4 — Spot the HTML noise

- **Question:** “Is the text dirty?”
 - **HINTS:**
 - Print 5 random reviews: `df['Review'].sample(5).tolist()`.
 - Look for: `&` (means `&`), `'` (means `'`), weird quotes `"`, line breaks `\n`.
 - **What you learn:** Yes, the text is dirty. You will clean it in Phase B.
-

Phase B — Clean the reviews (45 minutes)

Step 1 — Load the file

- **WHAT:** Load the CSV into a DataFrame called `df`.
- **HINTS:** `pd.read_csv('data/tripadvisor_hotel_reviews.csv')`.
- **EXPECTED:** Shape (20491, 2).

Step 2 — Look at the file

- **WHAT:** Check `.shape`, `.info()`, `.head()`, and `.isna().sum()`.
- **HINTS:** Just call them in different cells.
- **EXPECTED:**
 - 2 columns: Review (object / text), Rating (int).
 - Missing values: 0 (this dataset is complete — lucky for you).

Step 3 — Add a review_id column

- **WHAT:** Every row needs a unique ID. You will use it later.
- **HINTS:**
 - Use `df.reset_index(drop=True)`.
 - Then `df['review_id'] = 'r_' + df.index.astype(str)`.
- **WHY:** When you join tables in Module 5 / 7, you need a key.
- **EXPECTED:** A new column with values like `r_0`, `r_1`, ..., `r_20490`.

Step 4 — Fix the HTML entities

- **WHAT:** Replace `&` with `&`, `'` with `'`, `"` with `"`, etc.
- **HINTS:**
 - Python has a built-in helper: `import html` and then `html.unescape(text)`.
 - Apply it to every row: `df['Review'] = df['Review'].apply(__)`.
 - Fill in the blank with the function name.
- **WHY:** The text `it's` should become `it's`. The model will treat them as different words otherwise.
- **EXPECTED:** Print 5 reviews again. The `&` and `'` are gone.

Step 5 — Strip and normalize whitespace

- **WHAT:** Remove leading/trailing spaces. Replace `\n` and `\t` with a single space. Replace multiple spaces with one space.
- **HINTS:**
 - Skeleton:

```
import re
def clean_whitespace(text):
    text = text.replace('\n', ' ').replace('\t', __)
    text = re.sub(r'\s+', __, text)
    return text.__()      # strip
df['Review'] = df['Review'].apply(__)
```
 - Fill in the 4 blanks.
- **WHY:** A space is a space is a space. The model should not see "good hotel" and "good hotel" as different.

Step 6 — Lowercase a copy (KEEP the original!)

- **WHAT:** Make a new column `review_lower` with everything in lowercase.
- **HINTS:** `df['review_lower'] = df['Review'].str.lower()`.
- **WHY KEEP BOTH?**
 - The lowercased version is for counting words ("WIFI", "Wifi", "wifi" are the same thing).

- The original is for counting CAPS — angry reviewers often shout in capitals (“THIS HOTEL IS HORRIBLE!”). You will use this in Class 3.

Step 7 — Drop empty and very short reviews

- **WHAT:** Drop rows where the review is empty or has less than 10 characters.
- **HINTS:**
 - Make a mask: `mask = df['Review'].str.len() >= 10`.
 - Apply it: `df = df[mask].copy()`.
- **WHY:** “Great!” or “Bad” tells you almost nothing. Drop them.
- **EXPECTED:** You lose ~20 rows. Now ~20,471.

Step 8 — Drop duplicate reviews

- **WHAT:** Drop rows where the Review text is exactly the same as another row.
- **HINTS:**
 - `df = df.drop_duplicates(subset=['Review']).copy()`.
- **WHY:** Same text = same training example twice = the model overfits.
- **EXPECTED:** You lose ~1,400 rows. Now ~19,000.

Step 9 — Write down what you did

In a markdown cell, write: - Starting rows: ~20,491 - After dropping empty/short: ~20,471 - After dropping duplicates: ~19,000 (your final number) - WHY for each step.

Step 10 — Save to Drive

- **WHAT:** Save the cleaned DataFrame as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/tripadvisor_lab/reviews_step1.parquet')`.

Phase C — Make ONE chart for Emma (15 minutes)

She does not have time to read your code. She wants ONE picture.

Emma’s chart — “How unbalanced is the rating data?”

Make a bar chart showing the count of reviews per star rating (1 to 5).

- **HINTS:**
 - Use `df['Rating'].value_counts().sort_index().plot.bar()`.
 - On top of each bar, write the percentage. Use `plt.text(...)` in a loop.
 - Title: “50% of reviews are 5-star — only 5% are 1-star”.
 - X-label: Star rating.
 - Y-label: Number of reviews.
- **Takeaway for Emma:** “Most guests are happy. But that means our model will see very few unhappy reviews. We must handle this imbalance in Module 4 (using `class_weight='balanced'`).”

Common mistakes in Class 1

Mistake	What goes wrong
Lowercase the original column directly	You lose the CAPS information forever.
Forget <code>html.unescape</code>	Words like <code>it's</code> stay broken.

Mistake	What goes wrong
Forget <code>.copy()</code> after a filter Drop duplicates BEFORE adding <code>review_id</code>	Pandas warns “SettingWithCopyWarning”. The IDs in your file no longer match your raw line numbers.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ File loaded. Shape (20491, 2).
 - ☐ `review_id` column added.
 - ☐ HTML entities removed.
 - ☐ Whitespace normalized.
 - ☐ Lowercased copy `review_lower` added.
 - ☐ Empty / very short reviews dropped.
 - ☐ Duplicates dropped.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Emma.
 - ☐ You saved `reviews_step1.parquet` to your Drive folder.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Emma looks at your cleaned data. She is happy. But she says: “the model is a math model. The column Review is one big sentence. The model does not understand sentences. Turn the sentence into numbers.”

Your goal

Turn the text into NUMBERS. Start simple: counts. Length. Punctuation. Capital letters.

Inputs

- `reviews_step1.parquet` from Class 1

Outputs

- `reviews_step2.parquet` in Drive (with ~10 new numeric columns)
 - 3+ exploratory charts
 - 1 explanatory chart for Emma
-

Phase A — Explore the data first

Exploratory chart 1 — Review length by star rating

- **Question:** “Are angry reviewers longer or shorter than happy reviewers?”
- **HINTS:**
 - Use `sns.boxplot(x='Rating', y='review_length', data=df)`.
- **What you learn:** Often the 1-star and 2-star reviews are LONGER. Unhappy people write more.

Exploratory chart 2 — Histogram of review_length

- **Question:** “What is the typical length?”
- **HINTS:**
 - `plt.hist(df['review_length'], bins=60).`
 - Add `plt.xlim(0, 5000)` to ignore the few huge ones.
- **What you learn:** Most reviews are 200-1500 characters.

Exploratory chart 3 — Distribution of review_length AFTER `np.log1p`

- **Question:** “Does log-transform make the shape more normal?”
- **HINTS:**
 - Make a new column: `log_length = np.log1p(df['review_length']).`
 - Histogram it.
 - Compare to chart 2.
- **What you learn:** Log makes the long tail manageable for the model.

Exploratory chart 4 — Number of exclamation marks vs Rating

- **Question:** “Do happy reviews use more ! than angry reviews?”
 - **HINTS:**
 - First make `df['n_exclamation_marks'] = df['Review'].str.count('!').`
 - `df.groupby('Rating')['n_exclamation_marks'].mean().plot.bar().`
 - **What you learn:** Yes — 5-star reviews use many ! (“Amazing! Loved it! Will return!”). 1-star reviews use ! too but with anger. Interesting feature.
-

Phase B — Encode and scale

Step 1 — Word count and sentence count

- **WHAT:** Add 2 columns:
 - `n_words`: number of words.
 - `n_sentences`: number of sentences.
- **HINTS:**
 - For words: `df['n_words'] = df['Review'].str.split().str.len().`
 - For sentences: install `nlk` (you did this already), then:

```
from nltk.tokenize import sent_tokenize
df['n_sentences'] = df['Review'].apply(lambda t: len(sent_tokenize(__)))
```
 - Fill in the blank.
- **EXPECTED:** Most reviews have 5-30 sentences.

Step 2 — Average word length

- **WHAT:** `avg_word_length = (number of letters) / (number of words).`
- **HINTS:**
 - Skeleton:

```
def avg_word_len(text):
    words = text.__( )
    if len(words) == 0:
        return 0
    return sum(len(w) for w in words) / len(__( ))
```
 - `df['avg_word_length'] = df['Review'].apply(__( ))`
 - Fill in the 3 blanks.
- **WHY:** Longer words can be a sign of more careful or formal language.

Step 3 — Number of unique words

- **WHAT:** `n_unique_words` = how many DIFFERENT words are in the review.
- **HINTS:**
 - `df['n_unique_words'] = df['review_lower'].apply(lambda t: len(set(t.split()))).`
- **WHY:** A long review with only 50 unique words is more repetitive than a long review with 200 unique words.

Step 4 — Punctuation counts

Make these columns:

New column	What it counts
<code>n_exclamation_marks</code>	Number of ! in the review
<code>n_question_marks</code>	Number of ? in the review

- **HINTS:**
 - `df['n_exclamation_marks'] = df['Review'].str.count('!')`
 - `df['n_question_marks'] = df['Review'].str.count('\?')`
 - NOTE: ? is a special regex character. So you must write `\?`.

Step 5 — Capital letters features

Make these columns from the ORIGINAL Review (NOT the lowercased one):

New column	What it counts
<code>n_capital_words</code>	Number of WORDS that are all UPPERCASE (length >= 2)
<code>caps_ratio</code>	Number of capital letters / total letters

- **HINTS:**
 - Skeleton for `n_capital_words`:

```
def count_caps_words(text):  
    return sum(1 for w in text.split() if len(w) >= 2 and w.____())  
df['n_capital_words'] = df['Review'].apply(____)
```
 - For `caps_ratio`: count uppercase letters with a comprehension `sum(1 for c in text if c.isupper())`. Divide by total letters.
- **WHY:** “THE WORST HOTEL EVER!!!” is a strong signal for 1-star. The model needs the count.

Step 6 — Log-transform review_length and n_words

- **WHAT:** Both have very long tails. Apply `np.log1p()`.
- **HINTS:**
 - `df['log_length'] = np.log1p(df['review_length'])`.
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros.

Step 7 — Encode the target column Rating (preview)

- **WHAT:** Rating is already a number (1 to 5). For now, do nothing.
- **WHY think about it?**
 - For Module 4 you have a choice:
 - * Treat as **multi-class** (5 classes). Predict the most likely class.
 - * Treat as **regression**. Predict 3.7 stars. Round if you must.
 - Both are valid. Talk to your team. Write your choice in markdown.

Step 8 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler.`
 - `scaler = StandardScaler().`
 - `scaler.fit_transform(df[numeric_cols]).`
- **WARNING:** This is for inspection only. In Class 5, scaling goes **INSIDE** the Pipeline. Do **NOT** save the scaled version as your final file.

Step 9 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/tripadvisor_lab/reviews_step2.parquet')`
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Emma

Emma's chart — “Do angry guests SHOUT in capitals?”

A bar chart showing the average `caps_ratio` for each star rating.

- **HINTS:**
 - `df.groupby('Rating')['caps_ratio'].mean().plot.bar().`
- **Title:** “Angry guests use 3x more CAPITAL LETTERS than happy guests.”
- **X-label:** Star rating.
- **Y-label:** Average ratio of capital letters.
- **Takeaway:** “When the model sees a review full of caps, it should suspect 1-star. This feature alone is a strong signal.”

Common mistakes in Class 2

Mistake	What goes wrong
Compute <code>caps_ratio</code> on the lowercased text	You get 0 every time. Use the original.
Forget <code>\\?</code> in <code>str.count</code>	Regex error. Or counts everything.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
Forget <code>np.log1p</code> and use <code>np.log</code> on zeros	<code>np.log(0) = -inf</code> . Crash.

Self-check before Class 3

- ☐ `n_words`, `n_sentences`, `avg_word_length`, `n_unique_words` exist.
- ☐ `n_exclamation_marks`, `n_question_marks` exist.
- ☐ `n_capital_words`, `caps_ratio` exist.
- ☐ `log_length` exists.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Emma.
- ☐ `reviews_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

Scenario reminder: Emma says: “Counts and lengths are nice. But Emma wants the DASHBOARD: ‘this week, 12 reviews complained about wifi, 8 about breakfast.’ So you must MAKE columns that detect topics and emotion. This is the heart of the project.”

Your goal

Make NEW columns from the text. Topics. Sentiment. Negation. These are the columns the model will love.

Inputs

- reviews_step2.parquet

Outputs

- reviews_step3.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Emma
-

Phase A — Explore the data first

Exploratory chart 1 — Most common words in 1-star reviews

- **Question:** “What words appear in angry reviews?”
- **HINTS:**
 - Filter: `bad = df[df['Rating'] == 1]`.
 - Join all the lowercased text: `' '.join(bad['review_lower'])`.
 - Split into words, count, take top 30 (ignore common words like “the”, “and”).
 - Use `from collections import Counter`.
 - Or use a word cloud: `from wordcloud import WordCloud`.
- **What you learn:** Words like “room”, “dirty”, “smell”, “rude”, “wifi”, “noise” will appear a lot.

Exploratory chart 2 — Most common words in 5-star reviews

- Same as chart 1 but for `df['Rating'] == 5`.
- **What you learn:** “great”, “love”, “perfect”, “staff”, “clean”, “wonderful”.

Exploratory chart 3 — mentions_wifi rate by star rating

- After Step 3 below, plot:
 - `df.groupby('Rating')['mentions_wifi'].mean().plot.bar()`.
- **What you learn:** Wifi is mentioned mostly in 1-2 star reviews. The dashboard idea is real.

Exploratory chart 4 — VADER sentiment score vs star rating

- After Step 6 below, plot:
 - `sns.boxplot(x='Rating', y='vader_compound', data=df)`.
 - **What you learn:** The line goes up from 1-star to 5-star — VADER works.
-

Phase B — Engineer the features

Step 1 — Try to find a year (if possible)

This dataset does NOT have a clear date column. BUT some reviews mention a year like “stayed in 2015” or “visited Aug 2017”.

- **WHAT:** Try to extract the year from the text. If not found, leave as missing.
- **HINTS:**
 - Use a regex: `r'\b(20[0-2][0-9])\b'`.
 - Skeleton:

```
import re
def find_year(text):
    m = re.search(r'\b(20[0-2][0-9])\b', __)
    return int(m.group(1)) if m else None
df['review_year'] = df['Review'].apply(__)
```
 - Fill in the blanks.
- **EXPECTED:** Around 30-40% of reviews have a year. The rest are NaN. That is OK.
- **WHY:** Hotel quality changes over time. The year can help the model.

Step 2 — Build a topic dictionary

Emma wants the dashboard to know **which topics are mentioned**. You will check 5 topics:

Topic	Keywords (lowercase)
mentions_clean	“clean”, “dirty”, “smell”, “stain”, “filthy”, “spotless”
mentions_breakfast	“breakfast”, “buffet”, “coffee”, “morning meal”
mentions_wifi	“wifi”, “internet”, “wi-fi”, “wi fi”, “connection”
mentions_noise	“noise”, “noisy”, “quiet”, “loud”, “soundproof”
mentions_location	“location”, “downtown”, “central”, “near”, “walking distance”

Plan: for each topic, the column is 1 if ANY of the keywords appear in `review_lower`, else 0.

Step 3 — Compute the 5 mentions columns

- **WHAT:** Make 5 new columns using the dictionary above.
- **HINTS:**
 - Skeleton:

```
topic_dict = {
    'mentions_clean': ['clean', 'dirty', 'smell', 'stain', 'filthy', 'spotless'],
    'mentions_breakfast': [],
    'mentions_wifi': [],
    'mentions_noise': [],
    'mentions_location': [],
}

def has_any(text, keywords):
    return int(any(kw in text for kw in __))

for col, kws in topic_dict.items():
    df[col] = df['review_lower'].apply(lambda t: has_any(__, __))
```
 - Fill in the blanks.
- **WHY:** These columns are the BACKBONE of Emma’s dashboard. The dashboard will sum each column per week: “this week, 12 reviews had `mentions_wifi == 1`.”

Step 4 — Negation detection

- **WHAT:** A column `has_negation` = 1 if any of "not", "no", "never", "nothing", "don't", "didn't", "won't" appears.
- **HINTS:**
 - Make a list of negation words.
 - Check if any appears in the lowercased text.
 - Skeleton:

```
neg_words = ['not ', ' no ', 'never', __, __, __]
df['has_negation'] = df['review_lower'].apply(
    lambda t: int(any(w in t for w in __))
)
```
- **WHY:** “The room was clean” vs “The room was NOT clean” mean opposite things. Negation is critical.

Step 5 — Lexicon-based polarity score

A **lexicon** is a dictionary of positive and negative words. We count how many of each appear.

- **WHAT:** Build a tiny lexicon yourself. Count words. Compute a score: $(\text{positive} - \text{negative}) / (\text{positive} + \text{negative} + 1)$.
- **HINTS:**
 - Skeleton:

```
positive_words = {'great', 'amazing', 'love', 'wonderful', 'perfect',
                  'clean', 'friendly', 'comfortable', 'excellent', 'best'}
negative_words = {'bad', 'horrible', 'dirty', 'rude', 'terrible',
                  'worst', 'awful', 'broken', 'noisy', 'cold'}

def polarity(text):
    words = text.split()
    pos = sum(1 for w in words if w in __)
    neg = sum(1 for w in words if w in __)
    return (pos - neg) / (pos + neg + 1)
df['polarity_lexicon_score'] = df['review_lower'].apply(__)
```
 - Fill in the 3 blanks.
- **EXPECTED:** Score between -1 (very negative) and +1 (very positive).

Step 6 — VADER sentiment score (ready-made library)

VADER is a famous sentiment tool for English. You do not need to train anything.

- **WHAT:** Add a column `vader_compound` = the VADER score for each review.
- **HINTS:**
 - Skeleton:

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
def vader_score(text):
    return analyzer.polarity_scores(__)['compound']
df['vader_compound'] = df['Review'].apply(__)
```
 - Fill in the blanks.
- **EXPECTED:** Score between -1 and +1. Takes ~30 seconds for 19,000 reviews.
- **WHY:** VADER understands “not good” (negation) better than your simple lexicon. Two scores are better than one — the model can compare them.

Step 7 — Sanity-check correlation

- **WHAT:** Check that your engineered features actually correlate with Rating.
- **HINTS:**
 - `df[['Rating', 'vader_compound', 'polarity_lexicon_score', 'caps_ratio', 'has_negation']].corr()`
 - Look at the Rating row.

- **EXPECTED:**
 - vader_compound correlation with Rating: ~+0.6 (strong positive).
 - caps_ratio correlation: negative.
 - has_negation correlation: negative.
- If your numbers look wrong, fix them BEFORE Class 4.

Step 8 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/tripadvisor_lab/reviews_step3.parquet')`.

Phase C — Make ONE chart for Emma

Emma’s chart — “What do guests complain about most?”

A horizontal bar chart showing the % of 1-star and 2-star reviews that mention each topic.

- **HINTS:**
 - Filter: `bad = df[df['Rating'] <= 2]`.
 - For each `mentions_*` column, compute `bad[col].mean()`.
 - Put in a Series. Sort. Bar chart (horizontal).
- **Title:** “Top complaints in 1-2 star reviews (% of reviews that mention each topic).”
- **X-label:** % of bad reviews mentioning topic.
- **Y-label:** Topic.
- **Takeaway for Emma:** “Cleanliness is the #1 complaint. Wifi is #2. This IS the dashboard you asked for. We just need to refresh it every week.”

Common mistakes in Class 3

Mistake	What goes wrong
Use original Review for keywords (not lowercased)	“WIFI” is missed because you search for “wifi”.
Use ' no ' without spaces around	You match the word “now” too. Wrong negation count.
Run VADER on a huge column without saving	If Colab disconnects, you lose 30 minutes. Save often.
Forget that review_year is mostly NaN	Don’t drop the column — just leave the NaNs for the imputer in Class 5.
Use mentions_* columns as the only features	The text has MUCH more signal. These are just for the dashboard.

Self-check before Class 4

- ☐ review_year attempted. Most rows are NaN, some have a year.
- ☐ 5 mentions_* columns exist (0/1 each).
- ☐ has_negation exists.
- ☐ polarity_lexicon_score between -1 and +1.
- ☐ vader_compound between -1 and +1.
- ☐ vader_compound correlates positively with Rating.
- ☐ 3+ exploratory charts.
- ☐ 1 explanatory chart.
- ☐ reviews_step3.parquet saved to Drive.

Class 4 — Feature Selection

Scenario reminder: Now you have ~22 columns. Emma says: “Some columns are duplicates. Some are useless. I want the BEST 15-18 columns for the model. Pick them. Tell me why.”

Your goal

Pick the best columns. Drop the rest. Justify every choice.

Inputs

- reviews_step3.parquet

Outputs

- reviews_step4.parquet in Drive (only the selected columns + Rating + review_id + Review)
 - 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
 - A short markdown report explaining your choices
-

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - First make a list of numeric columns: `numeric_cols = df.select_dtypes(include='number').columns.tolist()`
 - Compute `df[numeric_cols].corr()`.
 - Plot with `sns.heatmap()`.
 - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** Pairs with `lccorr > 0.9` are redundant. Probably: `review_length` and `n_words` (almost the same thing). Probably: `log_length` and `review_length` too.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:** Sort the values, then bar chart.
- **What you learn:** Which columns predict Rating strongest. Expect `vader_compound` at the top.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop(['Rating', 'Review', 'review_lower', 'review_id'], axis=1).`
 - `y = df['Rating'].`

- Pass: `test_size=0.2, random_state=42, stratify=y`.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.
- **WHY drop Review?** It is text. The model in Module 4 will use TF-IDF on it separately. For now, the numeric features are the focus.

Step 2 — Handle missing review_year

- **WHAT:** Many rows have NaN for review_year. For now, fill with a sentinel: -1 (means “unknown”).
- **HINTS:** `X_train['review_year'] = X_train['review_year'].fillna(-1)`. Same for `X_test`.
- **WARNING:** In Class 5 the proper imputer will go inside the pipeline. This is just for ranking.

Step 3 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold`.
 - `VarianceThreshold(threshold=0.01)`. Then `.fit_transform(X_train_numeric)`.
- **WHAT TO LOOK FOR:** A mentions_* column with 99.9% zeros (almost no one mentions the topic) is a candidate to drop.

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about Rating. Because Rating has 5 classes, use the classification version.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif`.
 - `mi = mutual_info_classif(X_train_numeric, y_train, random_state=42)`.
 - Put in a Series, sort.
- **EXPECTED:** vader_compound, polarity_lexicon_score, caps_ratio, has_negation should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF. Look at feature_importances_.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier`.
 - `RandomForestClassifier(n_estimators=80, max_depth=10, n_jobs=-1, random_state=42)`.
 - `.fit(X_train_numeric, y_train)`.
 - Look at `.feature_importances_`.

Step 6 — Remove highly correlated columns

- **WHAT:** From any pair where `lccorr > 0.9`, drop the LESS important one (by RF importance).
- **HINTS:**
 - Compute correlation matrix.
 - For each pair `> 0.9`, compare RF importance of the two columns.
 - Drop the lower one.
- **TYPICAL OUTCOME:** Drop `n_words` (keep `review_length`). Drop `log_length` (keep `review_length`). Or the opposite — your choice.

Step 7 — Pick the final 15-18 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance / correlation pruning.

- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.
- **A GOOD SHORTLIST (example, not the answer):**
 - vader_compound, polarity_lexicon_score, caps_ratio, has_negation,
 - review_length, n_sentences, avg_word_length, n_unique_words,
 - n_exclamation_marks, n_question_marks, n_capital_words,
 - mentions_clean, mentions_breakfast, mentions_wifi, mentions_noise, mentions_location,
 - review_year.

Step 8 — Save

- **HINTS:** Keep selected columns + Rating + review_id + Review (you keep Review for Module 7 TF-IDF). Save as reviews_step4.parquet to Drive.

Phase C — Make ONE chart for Emma

Emma’s chart — “These are the 10 most important signals.”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - sort_values() then .head(10) then .plot.barh().
- **Title:** “Top 10 predictors of hotel rating.”
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.
- **Takeaway for Emma:** “Sentiment score (VADER) and caps ratio together explain most of the signal. Topic-mention columns are smaller but they POWER YOUR DASHBOARD — keep them all.”

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop the mentions_* columns because they have low MI	Emma’s DASHBOARD breaks. Keep them.
Forget to drop the text Review before MI	Crash: mutual_info_classif cannot handle strings.
Drop columns without writing why	Module 4 students will not understand.
Drop a high-mutual-info column	Big mistake. Check why it is high before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 15-18 columns remain + Rating + review_id + Review.
- ☐ You wrote down WHY for each kept column.
- ☐ mentions_* columns kept (Emma’s dashboard).
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Emma.
- ☐ reviews_step4.parquet saved to Drive.

Class 5 — Pipelines

Scenario reminder: Emma says: “Your cleaning code is in 4 different notebooks. When a new review arrives tomorrow, you cannot copy 4 notebooks to the server. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `reviews_step4.parquet` (selected columns)

Outputs

- `tripadvisor_pipeline.joblib` saved in Drive
 - 1 confusion-matrix chart (5x5 because 5 classes)
 - A pipeline that takes RAW input and produces predictions
-

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix. Because there are 5 classes (1, 2, 3, 4, 5), it is a 5x5 grid.
- **HINTS:**
 - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`
 - `cm = confusion_matrix(y_test, y_pred).`
 - `ConfusionMatrixDisplay(cm, display_labels=[1,2,3,4,5]).plot().`
- **What you learn:** Which star ratings are EASY to predict (probably 5-star) and which are HARD (probably 3-star, the in-between).

Exploratory chart 2 — Per-class F1 score

- After Step 6, look at `classification_report`.
 - **HINTS:** `from sklearn.metrics import classification_report` then print.
 - **What you learn:** F1 for 5-star is high. F1 for 1-star is medium. F1 for 3-star is LOW. This is normal.
-

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
 - numeric: `vader_compound, polarity_lexicon_score, caps_ratio, review_length, n_sentences, avg_word_length, n_unique_words, n_exclamation_marks, n_question_marks, n_capital_words, review_year.`
 - categorical (0/1, but treat as numeric is also OK): `has_negation, mentions_clean, mentions_breakfast, mentions_wifi, mentions_noise, mentions_location.`
- **NOTE:** Because all your columns are already numeric, you may not need a categorical branch at all. Your team can decide.

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - `from sklearn.pipeline import Pipeline.`
 - `from sklearn.impute import SimpleImputer.`
 - `from sklearn.preprocessing import StandardScaler.`
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')
- **WHY median?** `review_year` has many NaNs. The median is more stable than the mean.

Step 3 — Build the categorical mini-pipeline (optional)

- **WHAT:** If you separated the 0/1 mentions into a “categorical” group, fill missing with 0 (the most frequent), then leave them.
- **HINTS:**
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___', fill_value=0)),
])
```
 - Or just put them in the numeric branch — they are already 0/1.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - `from sklearn.compose import ColumnTransformer.`
 - It takes a list of tuples: (name, transformer, list_of_columns).
 - Skeleton:

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, ___),
    # ('cat', categorical_transformer, ___),    # optional
])
```
 - Fill in the column lists.

Step 5 — Add the model on top

- **HINTS:**
 - `from sklearn.linear_model import LogisticRegression.`
 - `LogisticRegression(class_weight='balanced', max_iter=2000, random_state=42, multi_class='auto').`
 - Put it in a Pipeline with the preprocessor.
- **WHY `class_weight='balanced'`?** Only 5% of reviews are 1-star. Without this, the model just predicts “5-star” for everyone and gets ~50% accuracy (but useless for Emma — she needs to find the bad reviews).

Step 6 — Train and evaluate

- **HINTS:**
 - `full_pipeline.fit(X_train, y_train).`
 - `y_pred = full_pipeline.predict(X_test).`
 - `from sklearn.metrics import classification_report.`
- **EXPECTED:** Macro F1 around 0.35-0.45. (Module 4 with TF-IDF improves this a lot.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/tripadvisor_lab/tripadvisor_pipeline.joblib')

Phase C — Make ONE chart for Emma

Emma’s chart — “How well does our baseline guess the rating?”

A clean confusion matrix (5x5) with star labels.

- **HINTS:**
 - ConfusionMatrixDisplay(cm, display_labels=['1 star','2 star','3 star','4 star','5 star']).plot(cmap='Blues').
- **Title:** “Baseline model — confusion matrix on 19,000 reviews.”
- **Takeaway for Emma:** “Our model is best at finding 5-star and 1-star reviews. It struggles with 3-star (the middle ones). Module 4 will fix this by reading the actual TEXT, not just our counts.”

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget class_weight='balanced'	Model predicts 5-star always. Useless for Emma.
Use class_weight='balanced' AND oversampling	Over-correction. Pick one.
Forget random_state	Results change every run. Hard to debug.
Include Review (text) in the pipeline input	Crash: scaler cannot scale text. Drop it first or add a TF-IDF branch.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Preprocessor has numeric (and maybe categorical) transformer.
- ☐ class_weight='balanced' set.
- ☐ Confusion matrix chart.
- ☐ Per-class F1 printed.
- ☐ tripadvisor_pipeline.joblib saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Emma is in the meeting room. She wants the FINAL clean dataset on her desk in 90 minutes. This is the lab.

Your goal

Take 1 raw CSV file. Produce ONE final .parquet file with the exact ~21-column schema. Plus a 1-page findings report for Emma.

Time

90 minutes of focused work.

Inputs

- `data/tripadvisor_hotel_reviews.csv` in your Drive folder

Outputs

- `tripadvisor_clean.parquet` (~19,000 rows x ~21 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. Rating distribution (the 50% / 5% imbalance problem). 2. Top complaints in 1-2 star reviews (Emma's dashboard preview). 3. Top 10 most important features.

These 3 charts tell Emma the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `tripadvisor_clean.parquet` MUST have these columns, with these names and types:

#	Column	Type	Source
1	<code>review_id</code>	string	engineered (Class 1)
2	<code>Review</code>	string	original cleaned text (kept for Module 7!)
3	<code>review_length</code>	int	engineered
4	<code>n_words</code>	int	engineered
5	<code>n_sentences</code>	int	engineered
6	<code>n_exclamation_marks</code>	int	engineered
7	<code>n_question_marks</code>	int	engineered
8	<code>n_capital_words</code>	int	engineered
9	<code>caps_ratio</code>	float	engineered
10	<code>avg_word_length</code>	float	engineered
11	<code>n_unique_words</code>	int	engineered
12	<code>mentions_clean</code>	int (0/1)	engineered (topic dict)
13	<code>mentions_breakfast</code>	int (0/1)	engineered (topic dict)
14	<code>mentions_wifi</code>	int (0/1)	engineered (topic dict)
15	<code>mentions_noise</code>	int (0/1)	engineered (topic dict)
16	<code>mentions_location</code>	int (0/1)	engineered (topic dict)
17	<code>has_negation</code>	int (0/1)	engineered
18	<code>polarity_lexicon_score</code>	float	engineered (lexicon)
19	<code>vader_compound</code>	float	engineered (VADER)
20	<code>review_year</code>	float (with NaN allowed)	engineered (regex)
21	<code>Rating</code>	int 1-5	TARGET (original)

Lab phases (timed)

Phase	Time	What you do
1. Load + IDs	5 min	Load CSV. Add review_id.
2. Clean text	15 min	HTML entities, whitespace, drop empty / duplicates, lowercased copy.
3. Basic counts	15 min	review_length, n_words, n_sentences, avg_word_length, n_unique_words, n_exclamation_marks, n_question_marks, n_capital_words, caps_ratio.
4. Topic mentions	10 min	The 5 mentions_* columns from the keyword dictionary.
5. Negation + lexicon + VADER	15 min	has_negation, polarity_lexicon_score, vader_compound.
6. Year + validate schema + save	5 min	review_year regex. Check column names. Save .parquet.
7. Findings	25 min	Three charts on one page. Write findings.md.

Total: 90 minutes.

Validation step (do this BEFORE you say “done”)

```
df = pd.read_parquet('tripadvisor_clean.parquet')
print(df.shape)           # ~19000 rows, ~21 columns
print(df.columns.tolist()) # match the schema above
print(df.dtypes)          # check types
print(df['Rating'].value_counts().sort_index()) # 1..5 present
print(df.isna().sum())    # only review_year should have NaN
```

If any line is wrong: fix it before submitting.

Phase C — Findings report for Emma (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: _____
- Number of columns: _____
- Most common rating: _____ (% _____)
- Least common rating: _____ (% _____)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

• _____

One chart that summarizes everything

Embed your most important chart (the “top complaints in 1-2 star reviews” one). Emma will put it on her dashboard.

Grading rubric (100 points)

Item	Points
All ~21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from raw CSV in ONE command)	10
VADER + lexicon scores are between -1 and +1	5
5 mentions_* columns all present and 0/1 only	10
Duplicates and very short reviews removed	5
At least 3 exploratory + 1 explanatory chart per class	20
findings.md has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/: - tripadvisor_clean.parquet - Your Colab notebook (File > Download > Download .ipynb) - findings.md

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate .parquet files to Drive** (reviews_step1, reviews_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Read 10 random reviews every class.** A model is only as good as the data you understand. Look at 5-star AND 1-star examples. Notice the patterns. Then turn the patterns into columns.
8. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.